

Chapitre 4 : Manipuler et nettoyer avec pandas

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Fil rouge** : les prix des denrées sur les marchés du Sénégal · **Version** : 2026-10-23

Fiche du chapitre

Objectifs (OS3)	A la fin du chapitre, l'apprenant sait : 1) charger un fichier CSV, Excel ou JSON dans un `DataFrame` ; 2) décrire un tableau (`shape` , `info` , `describe` , `value_counts`) ; 3) choisir, renommer et créer des colonnes ; 4) sélectionner des lignes avec `loc` , `iloc` et des conditions ; 5) trier ; 6) convertir des textes en nombres et en dates ; 7) repérer et traiter les valeurs manquantes et les doublons ; 8) enregistrer un tableau propre.
Prérequis	Chapitres 2 (les fichiers abîmés, en Python pur) et 3 (NumPy, masques)
Durée	🟢 Débutant : 8 h · 🔵 Standard : 6 h · 🟣 Expert : 4 h (atelier compris)
Matériel	Google Colab ; les relevés du PAM et le fichier d'enquête (déjà fournis) ; pour l'atelier, le classeur des relevés régionaux (fourni)
Livrable	Le notebook de l'atelier 4 : un classeur Excel de 4 537 relevés nettoyé, vérifié et documenté

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Ouvrez un nouveau notebook Colab, `chapitre4-pandas` , envoyez-y le fichier du PAM et le fichier d'enquête du chapitre 2, et tapez les exemples dans l'ordre.

---

1. pandas : un tableur dans Python

Au chapitre 2, il fallait une boucle et une fonction pour lire, convertir et filtrer un CSV ; au chapitre 3, NumPy calculait vite, mais sur des tableaux d'**un seul type**, sans noms de colonnes. **pandas** réunit les deux : un **tableau à colonnes nommées** (le `DataFrame`), où chaque colonne a son propre type, et des centaines d'outils pour le lire, le filtrer, le nettoyer et le résumer. Chaque colonne d'un `DataFrame` est une `Series`, construite sur un tableau NumPy.

Python pur (chapitre 2)	pandas (ce chapitre)
`csv.DictReader` et une liste de dictionnaires	`pd.read_csv(...)` : un `DataFrame`
Une boucle avec `if`	`df[condition]`
`float(...)` dans un `try`	`pd.to_numeric(..., errors="coerce")`
Un compteur dans un dictionnaire	`df["colonne"].value_counts()`

2. Charger et regarder

```
import pandas as pd

prix = pd.read_csv("wfp_food_prices_sen.csv")
print(prix.shape)
prix.head(3)
```

```
(52021, 16)
         date admin1 admin2  market  market_id  latitude  longitude           category
commodity  commodity_id unit priceflag pricetype currency   price  usdprice
0  2000-01-15  Dakar  Dakar  Tilene        447     14.68     -17.46  cereals and tubers  Maize
(imported)          158   KG    actual    Retail      XOF  149.00      0.35
1  2000-01-15  Dakar  Dakar  Tilene        447     14.68     -17.46  cereals and tubers
Millet            73   KG    actual    Retail      XOF  124.75      0.30
2  2000-01-15  Dakar  Dakar  Tilene        447     14.68     -17.46  cereals and tubers   Rice
(imported)           64   KG    actual    Retail      XOF  227.50      0.54
```

Morceau	Ce qu'il veut dire
`pd.read_csv(...)`	Lit le fichier, utilise la première ligne comme noms de colonnes, et **devine le type** de chaque colonne
`prix.shape`	(nombre de lignes, nombre de colonnes)
`prix.head(3)`	Les 3 premières lignes (`tail` pour les dernières). La colonne de gauche (0, 1, 2) est l'**index** : l'étiquette de chaque ligne

```
prix.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 52021 entries, 0 to 52020
Data columns (total 16 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   date          52021 non-null  str
 1   admin1        52021 non-null  str
 2   admin2        52021 non-null  str
 3   market        52021 non-null  str
 4   market_id     52021 non-null  int64
 5   latitude      52021 non-null  float64
 6   longitude     52021 non-null  float64
 7   category      52021 non-null  str
 8   commodity     52021 non-null  str
 9   commodity_id  52021 non-null  int64
 10  unit          52021 non-null  str
 11  priceflag     52021 non-null  str
 12  pricetype     52021 non-null  str
 13  currency      52021 non-null  str
 14  price         52021 non-null  float64
 15  usdprice      52021 non-null  float64
dtypes: float64(4), int64(2), str(10)
memory usage: 6.4 MB
```

`info()` est la **première commande** à lancer sur un nouveau tableau : le nombre de lignes, et pour chaque colonne le nombre de valeurs **non manquantes** et le **type**. Ici, aucune valeur manquante, et `price` a été lue directement comme un nombre (`float64`) : `read_csv` a fait la conversion du chapitre 2 tout seul. La date, elle, est restée un texte (`str`).

> 🔵 **Standard : `str` ou `object` ?** Avec pandas 3, une colonne de textes a le type `str` . Avec pandas 2 (encore utilisé par certaines versions de Colab), le même tableau affiche `object` . C'est la même chose pour vous : vérifiez votre version avec `pd.__version__` .

```
prix[["price", "usdprice", "latitude"]].describe().round(1)
```

```
          price  usdprice  latitude
count   52021.0   52021.0   52021.0
mean     1016.8       1.8      14.4
std     15540.5      27.3       1.0
min        55.0       0.1      12.5
25%       220.0       0.4      13.8
50%       275.0       0.5      14.5
75%       388.0       0.7      14.9
max    646500.0    1138.6      16.5
```

`describe()` donne en une ligne les statistiques du chapitre 3. Remarquez la moyenne de `price` (1 016,8 F) bien plus haute que la médiane (275 F), et le maximum à 646 500 F : les bovins vendus par tête (chapitre 2, section 5) tirent tout vers le haut.

3. Choisir et renommer les colonnes

```
prix = prix[["date", "admin1", "market", "commodity", "unit", "price"]]
prix = prix.rename(columns={"admin1": "region", "market": "marche", "commodity": "denree", "unit":
         "unite", "price": "prix"})
prix.columns.tolist()
```

```
['date', 'region', 'marche', 'denree', 'unite', 'prix']
```

Morceau	Ce qu'il veut dire
`prix[["date", ...]]`	Avec **une liste** de noms (doubles crochets) : un `DataFrame` réduit à ces colonnes
`rename(columns={ancien: nouveau})`	Renomme des colonnes ; renvoie un **nouveau** tableau, qu'on range à la place de l'ancien
`.columns.tolist()`	La liste des noms de colonnes

```
print(type(prix["prix"]), type(prix[["prix"]]))
prix["denree"].value_counts().head(4)
```

```
<class 'pandas.Series'> <class 'pandas.DataFrame'>
denree
Millet             9484
Rice (imported)    9308
Maize (local)      7489
Sorghum            6689
```

Un seul crochet (`prix["prix"]`) donne **une colonne** (`Series`) ; deux (`prix[["prix"]]`), un tableau d'**une** colonne. `value_counts()` compte les valeurs d'une colonne, de la plus fréquente à la moins fréquente : c'est le `Counter` du cours de Python, en une ligne.

> 🧨 **Essayez** : écrivez `prix["price"]` (l'ancien nom). Message : `KeyError: 'price'` : la colonne a été renommée. Les noms de colonnes sont sensibles aux majuscules et aux espaces. Annulez.

4. Sélectionner des lignes

4.1 Par position ou par étiquette

```
print(prix.iloc[0])
print(prix.iloc[0:3, 2:4])
print(prix.loc[2, "prix"])
```

```
date            2000-01-15
region              Dakar
marche             Tilene
denree    Maize (imported)
unite                  KG
prix                149.0
Name: 0, dtype: object
   marche            denree
0  Tilene  Maize (imported)
1  Tilene            Millet
2  Tilene   Rice (imported)
227.5
```

Morceau	Ce qu'il veut dire
`iloc[0]`	La ligne à la **position** 0 (*integer location*) ; elle s'affiche comme une `Series`
`iloc[0:3, 2:4]`	Les lignes 0 à 2 et les colonnes 2 et 3, **par position**, comme avec NumPy
`loc[2, "prix"]`	La ligne dont l'**étiquette** d'index est 2, colonne `prix`

`iloc` utilise les positions (0, 1, 2…), `loc` les étiquettes (l'index et les noms de colonnes). Tant que l'index est 0, 1, 2…, les deux se ressemblent ; après un filtre ou un tri, ils diffèrent (section 5).

4.2 Par conditions

```
riz_dakar_2025 = prix[(prix["denree"] == "Rice (imported)") & (prix["region"] == "Dakar") &
         (prix["date"].str.startswith("2025"))]
print(len(riz_dakar_2025), "relevés ; prix moyen", round(riz_dakar_2025["prix"].mean(), 1))
riz_dakar_2025.head(3)
```

```
22 relevés ; prix moyen 458.5
            date region    marche           denree unite   prix
44723  2025-01-15  Dakar    Tilene  Rice (imported)    KG  400.0
44732  2025-01-15  Dakar  Thiaroye  Rice (imported)    KG  400.0
45101  2025-02-15  Dakar    Tilene  Rice (imported)    KG  400.0
```

Le même résultat qu'aux ateliers 2 et 3, en une ligne. Les conditions sont les **masques** du chapitre 3 : `&`, `|`, `~`, et des parenthèses autour de chaque condition. `.str.startswith(...)` applique `startswith` à chaque texte de la colonne (toutes les méthodes des textes sont disponibles avec `.str.`).

> 🪛 **Essayez** : retirez les parenthèses : `prix[prix["region"] == "Dakar" & prix["unite"] == "KG"]`. Message : `TypeError: unsupported operand type(s) for &: 'str' and 'StringArray'` : `&` est calculé avant `==`. Annulez.

```
cereales = prix[prix["denree"].isin(["Millet", "Sorghum", "Maize (local)"])]
print(len(cereales))
print(cereales["denree"].unique().tolist())
```

```
23662
['Millet', 'Sorghum', 'Maize (local)']
```

`isin([...])` : la valeur fait partie de la liste. `unique().tolist()` : les valeurs différentes, en liste.

5. Trier

```
prix.sort_values("prix", ascending=False).head(3)
```

```
            date    region        marche              denree unite      prix
51453  2026-07-15  Diourbel  Ndoumbé DIOP  Livestock (cattle)  Head  646500.0
50514  2026-06-15  Diourbel        Bambey  Livestock (cattle)  Head  645000.0
51410  2026-07-15  Diourbel        Bambey  Livestock (cattle)  Head  642500.0
```

`sort_values("colonne", ascending=False)` trie du plus grand au plus petit ; plusieurs colonnes : `sort_values(["region", "prix"])`. Regardez l'index (51453, 50514...) : chaque ligne garde son **étiquette** d'origine. `loc[51453]` désigne toujours le bovin de Ndoumbé DIOP ; `iloc[0]`, la première ligne **après** le tri.

6. Nouvelles colonnes et dates

```
prix = prix[prix["unite"] == "KG"].copy()
prix["prix_euros"] = (prix["prix"] / 655.957).round(3)
prix["date"] = pd.to_datetime(prix["date"])
prix["annee"] = prix["date"].dt.year
print(prix.dtypes)
prix.tail(2)
```

```
date          datetime64[us]
region                   str
marche                   str
denree                   str
unite                    str
prix                 float64
prix_euros           float64
annee                  int32
dtype: object
           date      region      marche       denree unite   prix  prix_euros  annee
52019 2026-07-15  Ziguinchor  Saint-Maur     Tomatoes    KG  800.0       1.220   2026
52020 2026-07-15  Ziguinchor  Saint-Maur  Watermelons    KG  600.0       0.915   2026
```

Morceau	Ce qu'il veut dire
`prix[prix["unite"] == "KG"].copy()`	On écarte le bétail ; `.copy()` crée un **vrai** nouveau tableau, qu'on pourra modifier sans ambiguïté
`prix["prix_euros"] = ...`	Crée une colonne, calculée sur toute la colonne `prix` d'un coup

Morceau	Ce qu'il veut dire
`pd.to_datetime(colonne)`	Convertit des textes en **dates** (type `datetime64`)
`.dt.year`	L'année de chaque date ; aussi `.dt.month`, `.dt.day`, `.dt.day_name()`…

Avec de vraies dates, les comparaisons deviennent naturelles :

```
print(prix["date"].min(), prix["date"].max())
prix[prix["date"] >= "2026-01-01"]["annee"].value_counts()
```

```
2000-01-15 00:00:00 2026-07-15 00:00:00
annee
2026    3448
```

> 🟢 **Débutant : pourquoi `.copy()` ?** Un filtre comme `prix[prix["unite"] == "KG"]` peut donner un tableau qui « regarde » encore le tableau d'origine. Le modifier ensuite (créer une colonne) est ambigu : modifie-t-on l'original ? Avec pandas 2, cela déclenche l'avertissement `SettingWithCopyWarning`. Ajoutez `.copy()` dès que vous comptez modifier un tableau filtré.

7. Nettoyer le fichier d'enquête

Reprenons le fichier d'enquête abîmé du chapitre 2, avec pandas :

```
enquete = pd.read_csv("releves_enquete.csv")
print(enquete.shape)
print(enquete.dtypes)
enquete["prix"].head(3)
```

```
(300, 6)
date      str
region    str
marche    str
denree    str
unite     str
prix      str
dtype: object
0    300
1    500
2    950
```

La colonne `prix` est restée du **texte** (`str`) : une seule valeur illisible (`n.d.`, `387,5`) suffit pour que `read_csv` renonce à convertir toute la colonne. Et le calcul échoue :

```
try:
    enquete["prix"].mean()
except TypeError as e:
    print("Erreur :", str(e)[:90])
```

```
Erreur : Cannot perform reduction 'mean' with string dtype
```

7.1 Convertir les nombres

```
essai = pd.to_numeric(enquete["prix"], errors="coerce")
print(essai.isna().sum(), "prix illisibles ou manquants")
enquete.loc[essai.isna(), "prix"].unique().tolist()[:8]
```

```
17 prix illisibles ou manquants
[nan, '300,5', '420,5', '333,33', '1050,5', '1000,5', 'n.d.', '400,5']
```

Morceau	Ce qu'il veut dire
`pd.to_numeric(colonne, errors="coerce")`	Convertit en nombres ; ce qui ne se convertit pas devient `NaN` (manquant) au lieu de provoquer une erreur
`.isna()`	`True` pour chaque valeur manquante
`enquete.loc[masque, "prix"]`	`loc` accepte aussi un **masque** pour les lignes : les prix qui n'ont pas pu être convertis

On voit les coupables : les virgules décimales, `n.d.`, et les cases vides (`nan`). Corrigeons les virgules avant de convertir :

```
enquete["prix"] = pd.to_numeric(enquete["prix"].str.replace(",", "."), errors="coerce")
print(enquete["prix"].dtype, enquete["prix"].isna().sum(), "manquants")
```

```
float64 9 manquants
```

> 🏏 **Essayez** : `pd.to_numeric(enquete["prix"])`, sans `errors="coerce"`. Message : `ValueError: Unable to parse string "..." at position ...` : la conversion s'arrête à la première valeur illisible. `errors="coerce"` est l'équivalent pandas du `try` / `except` du chapitre 2.

7.2 Convertir les dates

Certaines dates sont au format français (`15/02/2025`). On essaie les deux formats, et on garde celui qui marche :

```
iso = pd.to_datetime(enquete["date"], format="%Y-%m-%d", errors="coerce")
fr = pd.to_datetime(enquete["date"], format="%d/%m/%Y", errors="coerce")
enquete["date"] = iso.fillna(fr)
print(enquete["date"].isna().sum(), "dates illisibles")
```

```
0 dates illisibles
```

Morceau	Ce qu'il veut dire
`format="%Y-%m-%d"`	Le format attendu : année, mois, jour (`%d/%m/%Y` pour jour/mois/année)
`errors="coerce"`	Une date qui ne correspond pas au format devient `NaT` (*not a time*, la date manquante)
`iso.fillna(fr)`	Là où `iso` est manquant, on prend la valeur de `fr`

8. Les valeurs manquantes

```
enquete.isna().sum()
```

```
date      0
region    0
marche    0
denree    0
unite     0
prix      9
```

`isna().sum()` compte les valeurs manquantes **par colonne**. Il reste 9 prix manquants (les cases vides, les `n.d.`, et les lignes qui avaient une colonne en moins).

```
print((enquete["prix"] <= 0).sum(), "prix négatifs ou nuls")
print(enquete["unite"].value_counts().to_dict())
```

```
2 prix négatifs ou nuls
{'KG': 297, 'sac': 3}
```

On garde les relevés valides : prix positif, unité `KG`, et aucune case vide :

```
propre = enquete[(enquete["prix"] > 0) & (enquete["unite"] == "KG")].dropna()
print(len(propre), "relevés gardés sur", len(enquete))
```

```
286 relevés gardés sur 300
```

286 : exactement le résultat de l'atelier 2, obtenu en Python pur avec une fonction de 20 lignes et une boucle. `dropna()` supprime les lignes qui ont **au moins une** valeur manquante ; `dropna(subset=["prix"])` ne regarde que la colonne `prix`.

8.1 Remplacer plutôt que supprimer ?

Supprimer une ligne perd de l'information. On peut aussi **remplacer** la valeur manquante, par exemple par la médiane des prix de la même denrée :

```
copie = enquete.copy()
mediane = copie.groupby("denree")["prix"].transform("median")
copie["prix_complete"] = copie["prix"].fillna(mediane)
print(copie["prix"].isna().sum(), "->", copie["prix_complete"].isna().sum())
copie[copie["prix"].isna()][["denree", "prix", "prix_complete"]].head(3)
```

```
9 -> 0
                           denree  prix  prix_complete
61   Rice (ordinary, first quality)   NaN          545.0
89                     Rice (local)   NaN          420.0
134               Maize (imported)   NaN          300.0
```

Morceau	Ce qu'il veut dire
`copie.groupby("denree")["prix"]`	Les prix, groupés par denrée (le chapitre 5 détaillera `groupby`)
`.transform("median")`	Pour **chaque ligne**, la médiane de **son** groupe : une colonne de même longueur que le tableau
`.fillna(mediane)`	Remplace chaque `NaN` par la valeur correspondante

> 🔵 **Standard : supprimer ou remplacer ?** Remplacer un prix manquant par la médiane donne un tableau complet, mais **invente** une valeur : les moyennes deviennent un peu trop « sages ». Règle : **supprimez** quand il y a peu de valeurs manquantes et beaucoup de données ; **remplacez** seulement quand c'est indispensable (un modèle qui refuse les `NaN`, par exemple), en le disant. Dans tous les cas, gardez une trace de ce que vous avez fait.

9. Les doublons

Une ligne recopiée deux fois fausse les comptages et les moyennes. Le fichier d'enquête n'en a pas : fabriquons-en trois pour l'exemple :

```
double = pd.concat([propre, propre.head(3)])
print(len(double), double.duplicated().sum(), "doublons")
sans = double.drop_duplicates()
print(len(sans))
```

```
289 3 doublons
286
```

Morceau	Ce qu'il veut dire
`pd.concat([a, b])`	Met deux tableaux bout à bout (chapitre 5)
`.duplicated()`	`True` pour chaque ligne **identique** à une ligne précédente
`.drop_duplicates()`	Supprime ces lignes, en gardant la première ; `subset=[...]` pour ne comparer que certaines colonnes

10. Excel et JSON

pandas lit et écrit aussi les classeurs **Excel** et le **JSON** :

```
propre.to_excel("enquete_propre.xlsx", index=False)
relu = pd.read_excel("enquete_propre.xlsx")
print(relu.shape, relu["date"].dtype)
propre.head(2).to_json("enquete_extrait.json", orient="records", date_format="iso",
         force_ascii=False, indent=2)
with open("enquete_extrait.json", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
(286, 6) datetime64[us]
[
  {
    "date":"2025-08-15T00:00:00.000",
    "region":"Dakar",
    "marche":"Thiaroye",
    "denree":"Maize (local)",
    "unite":"KG",
    "prix":300.0
  },
  {
    "date":"2025-06-15T00:00:00.000",
    "region":"Diourbel",
    "marche":"Bambey",
    "denree":"Sorghum",
    "unite":"KG",
    "prix":500.0
  }
]
```

Morceau	Ce qu'il veut dire
`to_excel("...", index=False)`	Écrit un classeur Excel, sans la colonne d'index ; `pd.read_excel` le relit (dates comprises)
`sheet_name="..."`	Le nom de la feuille à lire ou écrire (par défaut, la première)
`to_json(..., orient="records")`	Une liste d'objets, un par ligne

Morceau	Ce qu'il veut dire
`date_format="iso"`, `force_ascii=False`	Dates au format standard, accents conservés

Format	Lire	Écrire
CSV	`pd.read_csv`	`df.to_csv(..., index=False)`
Excel	`pd.read_excel`	`df.to_excel(..., index=False)`
JSON	`pd.read_json`	`df.to_json(...)`

Exercices

Dans votre notebook du chapitre, avec le fichier du PAM (colonnes renommées comme en section 3, date convertie) :

- ★ **Exercice 1 : faire connaissance.** Combien de marchés, de denrées et de régions différents le fichier contient-il ? Indice : `.nunique()`. Attendu : 67, 29, 14. (Section 3.)
- ★ **Exercice 2 : le niébé à Kaolack.** Sélectionnez les relevés de niébé (`Beans (niebe)`) dans la région de Kaolack en 2023. Combien y en a-t-il ? Quel est le prix le plus bas, le plus haut ? Dans quel marché et à quelle date le niébé était-il le moins cher ? Attendu : 48 relevés, de 513 à 1 200 F. (Sections 4 et 5.)
- ★★ **Exercice 3 : la soudure.** Pour le mil, créez une colonne `mois` (`.dt.month`) et une colonne `soudure`, vraie de juin à septembre (`isin`). Comparez le prix moyen du mil pendant la soudure et le reste de l'année. Attendu : 253,9 F contre 239,2 F. (Section 6.)
- ★★ **Exercice 4 : un petit tableau sale.** Créez `pd.DataFrame({"etudiant": [" awa ", "AWA", "Moussa", "Moussa", "khady"], "note": ["12,5", "14", "abs", "abs", "17"]})`. Nettoyez les noms (`str.strip()`, `str.title()`), convertissez les notes, supprimez les doublons, puis calculez la moyenne de chaque étudiant. Que vaut celle de Moussa, et pourquoi ? (Sections 7 à 9.)
- ★★★ **Exercice 5 : les valeurs extrêmes, par denrée.** Sur les relevés au kilo de 2025, calculez pour chaque relevé un z-score **par rapport à sa propre denrée**, avec `groupby("denree")["prix"].transform("mean")` et `transform("std")`. Combien de relevés dépassent 3 en valeur absolue ? Affichez les trois plus élevés. Attendu : 26 relevés sur 3 686. (Sections 6 et 8.1 ; chapitre 3, atelier.)

Atelier 4 : nettoyer les relevés régionaux

Le travail principal du chapitre : les régions ont envoyé leurs relevés de 2024 dans un classeur Excel commun, rempli à la main. Vous y trouverez des lignes recopiées deux fois, des régions écrites de cinq façons, des prix en texte, des prix au sac de 50 kg au lieu du kilo, des cases vides et des dates en texte. Vous le nettoyez étape par étape, vous **vérifiez** votre nettoyage en le comparant aux relevés officiels du PAM, et vous enregistrez le classeur propre avec un bilan. L'énoncé, le notebook de départ et le classeur vous sont fournis séparément.

A retenir

- `pd.read_csv`, `pd.read_excel`, `pd.read_json` ; puis **toujours** `shape`, `head`, `info`, `describe`.
- Une colonne : `df["nom"]` (`Series`) ; plusieurs : `df[["a", "b"]]` ; `rename(columns={...})`.
- `iloc` par **position**, `loc` par **étiquette** (et masques) ; conditions avec `&`, `|`, `~` entre parenthèses ; `isin`, `.str.startswith`…
- `sort_values` ; l'index garde l'étiquette d'origine des lignes.
- `pd.to_datetime` (avec `format=`) et `.dt.year`, `.dt.month` ; `pd.to_numeric(..., errors="coerce")` ; `.str.replace` avant de convertir.
- Manquants : `isna().sum()`, `dropna`, `fillna` (avec prudence) ; doublons : `duplicated`, `drop_duplicates`.
- `.copy()` avant de modifier un tableau filtré ; `index=False` en enregistrant.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`KeyError: 'price'`	Nom de colonne inexistant (renommé, faute de frappe, espace caché)	Vérifier `df.columns.tolist()`
`TypeError: Cannot perform reduction 'mean' with string dtype`	Une colonne de nombres lue comme du texte	`pd.to_numeric(..., errors="coerce")` après nettoyage
`ValueError: Unable to parse string "12,5" at position 0`	`to_numeric` sans `errors="coerce"` sur une valeur illisible	`.str.replace(",", ".")`, puis `errors="coerce"`
`TypeError: unsupported operand type(s) for &: 'str' and 'StringArray'`	Conditions sans parenthèses	`(df["a"] == x) & (df["b"] == y)`
`SettingWithCopyWarning` (pandas 2)	Modification d'un tableau filtré	`.copy()` après le filtre
Un filtre ne garde aucune ligne, sans erreur	Valeur mal écrite (`"Thiès"` au lieu de `"Thies"`), espaces, majuscules	`df["colonne"].unique().tolist()` pour voir les vraies valeurs
Les dates restent du texte, ou deviennent `NaT`	Format différent de celui donné	Vérifier `format=` ; plusieurs formats : essayer l'un puis l'autre, avec `fillna`
Les moyennes changent après `fillna`	Valeurs inventées	Le documenter ; préférer `dropna` quand c'est possible
`ModuleNotFoundError: No module named 'openpyxl'` (sur votre ordinateur)	Le module pour lire et écrire Excel manque	`pip install openpyxl` (il est déjà installé dans Colab)

Pour aller plus loin

- *10 minutes to pandas*, la prise en main officielle (en anglais) : https://pandas.pydata.org/docs/user_guide/10min.html
- Le guide des valeurs manquantes (en anglais) : https://pandas.pydata.org/docs/user_guide/missing_data.html
- *Python Data Science Handbook*, de Jake VanderPlas, chapitre 3 sur pandas, en accès libre (en anglais) : https://jakevdp.github.io/PythonDataScienceHandbook/